

Home

Library

Profile

Stories

Stats

Following

Python in Plain En...

Generative AI

Dev Genius

Level Up Coding

AI Quick Tips

AI Mind

Top Python Librari...

The Useful Tech

Realworld AI Use C...

All About ChatGPT

More

Member-only story

I Turned Andrej Karpathy's Autoresearch Into a Universal Skill



Balu Kosuri

Follow

12 min read · Mar 21, 2026



332



4



By Balasubramanyam Kosuri

I'm a technical writer. I spend my days in documentation repos, Markdown files, API references, style guides, and SEO audits. I don't train language models. I don't write CUDA kernels. But when Andrej Karpathy dropped his autoresearch, I couldn't stop thinking about it.

Andrej Karpathy's Autoresearch as a Skill

Here is my repository.

The idea was so simple it felt obvious in hindsight:

let an AI agent run experiments on its own, measure the results, keep what works, throw away what doesn't, and repeat until it's good. Karpathy was using it to train a small GPT model. The agent would modify the training code, run it for 5 minutes, check if the model got better, and decide — keep or revert. Then try something else. Over and over, autonomously, while he slept.

I remember reading the repo and thinking: this isn't for me.

And I didn't know how to apply it to my world. **Not yet.**

[btw: Andrej Karpathy released his open-source project **autoresearch** around **March 8–9, 2026**]

The YouTuber that helped me connect the dots

A few days after Karpathy's repo went public, I stumbled on a YouTube video that changed how I thought about it. The creator ([Nick Saraev](#)) had taken Karpathy's exact pattern and applied it to something completely different: optimizing a text-to-image prompt for generating whiteboard-style diagrams.

Watch the video: btw published on [Mar 14, 2026](#)

His setup was elegant:

- The thing being optimized: a prompt sent to Gemini's image generation model
- The measurement: Claude's vision API evaluating each generated diagram against 4 binary criteria
- The loop: generate 10 diagrams, grade them, keep the prompt if it scores higher, mutate it to fix failures, repeat every 2 minutes

His 4 criteria were dead simple:

1. Is all the text legible and grammatically correct? Yes/No
2. Are the colors soft pastels? Yes/No
3. Is the layout linear (left-to-right or top-to-bottom)? Yes/No
4. Are there any numbers or ordinals? Yes/No

10 diagrams, 4 questions each, max score of 40. He started at 32/40. By run 6 — about 12 minutes later — he hit 40/40. Perfect score.

What struck me wasn't the diagrams. It was the realization that the three ingredients needed — an objective metric, an automated measurement tool, and something to change — mapped perfectly to prompts:

Karpathy's AutoresearchPrompt Optimization `train.py` (code being modified) `prompt.txt` (prompt being modified) `val_bpb` (objective number) `Eval` score out of 40 (objective number) `evaluate_bpb()` (automated test) Claude vision grading against yes/no criteria Git keep/revert Keep best prompt / revert to best

The mapping was perfect. And if it worked for diagrams, it could work for anything with a measurable output. Including documentation.

Vibe-Coding it into a Skill.md

I decided to build it. Not just for diagrams — for everything. A universal autoresearch skill that could adapt to any repository, any tech stack, any optimization goal.

I opened Cursor, pulled up the Karpathy repo, read through the three files that mattered (`program.md` , `train.py` , `prepare.py`), rewatched the

video, and started prompting.

The first thing I got the AI to do was understand the pattern deeply. Not just “what files are in the repo” but the actual mechanics — the loop, the keep/discard logic, the mutation step, why binary evals matter, why you mutate from the best prompt and not the latest failed one.

Then I said: build me a skill that does this for any repo.

Version 1.1

The v1 skill had 5 phases:

1. **Repo Discovery** — scan the codebase, identify languages, frameworks, purpose
2. **Target Suggestions** — based on the scan, suggest what could be optimized (test quality, doc completeness, accessibility, SQL patterns — whatever fit the repo)
3. **Metric Definition** — auto-generate 4–6 binary yes/no eval criteria from industry best practices
4. **Baseline** — run the prompt once to establish a starting score
5. **Autoresearch Loop** — generate, evaluate, score, keep/discard, mutate, repeat autonomously

It worked. But it had gaps.

I used another AI to critique the skill.md

I ran the v1 skill through a rigorous analysis. These were what i missed in v1:

“The mutation strategy is underspecified.” Telling the agent to “analyze failures and rewrite the prompt” is too vague. The fix: define 6 explicit

mutation operators — add a constraint, add a negative example, restructure the prompt, tighten vague language, remove bloat, add a counterexample. Rotate through them so each gets tried. Log which one was used so you can see which mutation types are most productive.

“No validation set.” If you pick different items each cycle, score differences might reflect item difficulty rather than prompt quality. The fix: designate 3–5 fixed items that appear in every single cycle. Compare scores on this fixed set for apples-to-apples evaluation. Rotate the rest for coverage.

“The eval step conflates generator and judge.” Claude writes the prompt, generates outputs from it, and then judges those outputs — all in the same conversation. By the time it evaluates, it knows what the output was *trying* to do and grades charitably. The fix: evaluate in isolation. Present only the raw output and the criterion text. No prompt context. Judge as if seeing it for the first time.

“No handling of context window limits.” Over 20+ cycles, the conversation grows enormous. Early-cycle details fall off. The mutation step — which depends on understanding failure patterns — degrades silently. The fix: re-read all state from disk at the start of every cycle. Files are the source of truth, not conversational memory.

“Sample variance isn’t tracked.” “Vary the sample” with no memory of what was already sampled means you could run 20 cycles and accidentally never hit a particular file. The fix: track sampled items in `state.json` and do coverage-first selection — prefer untested items, only repeat after full coverage.

Each critique led to a specific, targeted fix. The skill got tighter with each round.

What the now version includes

The skill that emerged from this process has features the original diagram skill never had:

- **Eval isolation** — outputs are judged without knowledge of the prompt that produced them
- **Validation set** — fixed items for consistent comparison, rotating items for coverage
- **Structured mutation operators** — 6 named strategies, rotated and logged
- **Sample tracking** — coverage-first selection, no accidental gaps
- **Context window management** — re-reads state from disk every cycle
- **Command evals** — shell commands for deterministic checks (linting, compilation, grep) alongside LLM judgment
- **Item-level failure detection** — flags items that always fail (broken items, not bad prompts)
- **Adversarial re-eval** — every cycle, re-checks passing outputs with skeptical prompting
- **Criteria health check** — at run 10, flags criteria that are too easy or too hard
- **Plateau breaker** — after 5 stale runs, throws away the prompt structure and writes a fresh one from scratch using only the criteria and failure history
- **Confidence margin** — prevents noise from being mistaken for progress on small batches
- **Plan mode start** — scans and plans read-only before touching anything

A docs example: SFO for technical documentation

A good example: SEO for technical documentation

Since I'm a technical writer, let me walk through exactly how this skill would work on a documentation repo. Say I have a docs site built with Docusaurus, and I want to improve SEO compliance across my pages.

I open the repo and say:

```
Run autoresearch to optimize SEO for my documentation
```

Phase 1 — The agent scans:

```
Repo: my-product-docs
Stack: Markdown, MDX, Docusaurus, Node.js
Purpose: Technical documentation site for a SaaS API product
Quality tools found: markdownlint, broken-link-checker, Docusaurus build
```

Phase 2 — Skipped (I already stated my goal)

Phase 3 — The agent defines eval criteria:

```
Eval criteria for docs SEO compliance (8 pages x 5 criteria = max 40):
```

1. Does the page have a unique, descriptive <title> under 60 characters?
– yes/no – command: `grep -q "title:" [file]`
2. Does the page have a meta description between 120-160 characters?
– yes/no – command: `grep -q "description:" [file]`
3. Does the page use exactly one H1 heading?
– yes/no – command: `grep -c "^# " [file] | grep -q "^1$"`
4. Do all images have descriptive alt text (not empty, not "image", not "screenshot")?

– yes/no – llm-judge

5. Does the page have internal links to at least 2 other documentation pages?

– yes/no – llm-judge

I review. Looks good. I switch to Agent mode.

Phase 4 — Baseline:

The agent creates `.autoresearch/` and writes an initial prompt:

```
Generate SEO-optimized frontmatter and structural improvements for the following documentation page. Ensure the title is descriptive and under 60 characters. Include a meta description of 120-160 characters that summarizes the page content. The page should have exactly one H1 heading. All images must have descriptive alt text. Include internal links to related documentation pages.
```

Baseline score: 24/40

Breakdown:

- Title under 60 chars: 7/8
- Meta description: 4/8 (many pages missing or too short)
- Single H1: 8/8
- Alt text: 3/8 (generic “screenshot” on most images)
- Internal links: 2/8 (most pages are islands)

The loop runs:

Run 2 — Mutation: *add constraint*. Added “Meta descriptions MUST be between 120–160 characters. If the page discusses an API endpoint, the description must include the HTTP method and path.” Score: 28/40.

KEEP.

Run 4 — Mutation: *add negative example*. Added “DO NOT use generic alt text like ‘image’, ‘screenshot’, ‘diagram’, or ‘figure’. Instead describe what the image shows: ‘Authentication flow showing JWT token exchange between client and server’.” Score: 31/40. KEEP.

Run 6 — Mutation: *tighten language*. Changed “Include internal links to related pages” to “MUST contain at least 2 internal links to other documentation pages. Link text must be descriptive (not ‘click here’ or ‘see more’).” Score: 35/40. KEEP.

Run 9 — Mutation: *add counterexample*. Added a before/after example:

```
BAD: description: "This page covers authentication" (too short, 42 chars)
GOOD: description: "Configure OAuth 2.0 authentication for your API integra
```

Score: 38/40. KEEP.

Run 12 — Score: 40/40. KEEP. Perfect score. Run 13–40/40. Run 14–40/40. Three consecutive perfect scores. Loop stops.

Final result:

```
AUTORESEARCH COMPLETE
Runs: 14
Starting score: 24/40
Final best score: 40/40
Improvement: 66.7%
Runs kept: 6
Most effective mutation operators:
1. add_counterexample (2/2 KEEPs — 100%)
2. tighten_language (2/3 KEEPs — 67%)
3. add_constraint (1/2 KEEPs — 50%)
4. add_negative_example (1/2 KEEPs — 50%)
5. restructure (0/2 KEEPs — 0%)
6. remove_bloat (0/1 KEEPs — 0%)
```

```
Best prompt saved to: .autoresearch/best_prompt.txt  
Full history: .autoresearch/results.jsonl
```

The best prompt in `.autoresearch/best_prompt.txt` is now a battle-tested, optimized set of instructions for generating SEO-compliant documentation. I can use it as a template, feed it to other tools, or apply it across my entire docs site.

And I got here by doing nothing after the initial setup. The agent ran 14 cycles autonomously, figured out what worked, threw away what didn't, and handed me the winner.

Am i in the right direction?

I'm a **technical writer**. I didn't build a startup. I didn't train a model. I took an idea from one of the most respected AI researchers alive, watched a YouTuber translate it from ML training to prompt optimization, and vibe-coded it into something I can use in my own work — documentation SEO, style guide compliance, content quality, API reference completeness.

The skill may not be perfect. It will evolve. The eval isolation could be stronger. The mutation operators could be smarter. Future models will make the loop faster and the judgments more accurate. But the architecture is sound: scan, suggest, define metrics, run the loop, keep winners, throw away losers.

If you're reading this and thinking "I could use this for [my thing]" — you're right. That's the point. The skill is universal because the pattern is universal. Generate, evaluate, keep the best, try again.

This is the part that still amazes me. I didn't write a single line of the skill by hand. I described what I wanted in natural language, let the AI build it, ran it through two rounds of critique, and incorporated the fixes — all in one conversation. The AI wrote the skill. The AI critiqued the skill. The AI fixed the skill. And now the skill itself uses AI to optimize AI-generated outputs.

We've reached the point where a technical writer can take a concept from a world-class ML researcher, translate it through a YouTube video, and build production-grade tooling by having a conversation. No Python expertise required. No ML background. Just an idea and the ability to describe what you want.

This is how AI works today. Not next year. Today.

V2 — March 23, 2026

The day after publishing v1, I used the skill on a real project — optimizing SEO for our product documentation, specifically the front matter. And I ran straight into a problem the skill's design had created.

What went wrong

When I told the agent “optimize docs SEO,” the initial prompt it generated was all over the place. It tried to cover headings, alt text, internal links, meta descriptions, readability, keyword density — everything even loosely related to SEO. I didn't want all of that. I wanted it focused on front matter only, for our specific product docs. But the skill had no way to capture that level of specificity. It took my broad

target and ran with it in every direction.

The root cause: Phase 2 only asked what to improve. It never asked which part to focus on, or what constraints apply. So the initial prompt in Phase 4 was unfocused, and every cycle was fighting that unfocused foundation.

What changed

1. Three-field optimization template

Phase 2 now opens with a blank template before offering any suggestions:

Target: _____
(What do you want to improve? Pick something measurable.)

Scope: _____
(Which specific part? Narrow it down so the prompt stays focused.)

Context: _____
(Any constraints, conventions, or product-specific details the prompt should know.)

With examples that are developer-focused:

```
Target:  error handling      | test coverage      | API response va
Scope:  async service layer  | auth module unit tests | REST endpoint in
Context: uses Result<T> pattern | pytest + factory_boy  | OpenAPI 3.1, cus
```

For my docs SEO case, I would now fill in: Target = “SEO compliance”, Scope = “front matter only”, Context = “Docusaurus, SaaS API product, title/description/keywords fields.” The agent uses all three fields to write

a tightly scoped initial prompt — no tangents.

2. User-first flow

The old Phase 2 immediately generated suggestions and asked the user to pick one. The new flow flips this: it presents the blank template first and asks “What do you want to optimize today?” If the user already knows their goal, they fill it in and skip straight to metrics. Only if they say “suggest” does the agent generate suggestions. This puts the user in the driver’s seat instead of making them react to a pre-built list.

3. Universal quality dimensions replace hardcoded examples

The old Phase 2 had six hardcoded repo-type examples (Python API, React frontend, technical docs, SQL/database, CLI tool, DevOps/IaC). While the skill said “these are examples, not limits,” the specificity anchored the agent toward those categories. A user working on a mobile app, an ML pipeline, or game code wouldn’t see themselves in the list.

The new version replaces those examples with nine universal quality dimensions the agent scans through:

- Correctness, Testing, Performance, Security, Maintainability, Observability, Reliability, Developer Experience, Compliance and Standards

For each dimension, the agent evaluates whether it’s relevant to the scanned repo and generates specific, grounded suggestions. It discards irrelevant dimensions (no “accessibility” suggestions for a pure backend data pipeline) and keeps the top 5–8 most impactful ones. The skill also explicitly says: “The dimensions are a checklist to ensure breadth, not a cage” — so the agent can suggest things outside the framework if it finds them.

4. “Your own idea” as a visible option

When the agent does generate suggestions, the last numbered item is always “Your own idea — describe any optimization goal in your own words.” This was previously buried in a footnote that was easy to miss. Now it’s a first-class option in the list.

5. Scope and context persist across the loop

The `state.json` file now stores `scope` and `context` alongside `target`. This means when the agent re-reads state from disk at the start of every cycle (which it must — conversational memory is unreliable), it has the full picture of what the user asked for. The prompt generation in Phase 4 explicitly says: "If the user provided scope and context, use them to keep the prompt tightly focused — do not broaden beyond what the user asked for."

Why

The v1 skill optimized prompts well once it got going. The problem was the starting point. A broad, unfocused initial prompt means the loop spends its first several cycles just narrowing scope instead of improving quality. That’s wasted runs.

V2 fixes the input side. By capturing `target`, `scope`, and `context` upfront, the initial prompt starts focused, and the loop can spend every cycle making the prompt better rather than trying to figure out what the prompt should even be about.

The architecture is the same — scan, suggest, define metrics, run the loop, keep winners, throw away losers. V2 just makes sure the loop starts from the right place.

Try It Yourself

The skill files are in this folder:

- Cursor: copy `cursor/SKILL.md` to `~/.cursor/skills/autoresearch-universal/SKILL.md`
- Claude Code: copy `claude-code/SKILL.md` to `~/.claude/skills/autoresearch-universal/SKILL.md`

Open any repo. Switch to Plan mode. Say “run autoresearch” See what happens.

If you want to go deeper, read the `SKILL.md` itself — it's 400 lines of plain English that tell the agent exactly what to do, step by step.

If you're a technical writer like me, start with docs SEO. If you're a front-end dev, start with accessibility. If you're a backend engineer, start with test case quality. If you're a DBA, start with query patterns.

The skill doesn't care what you optimize. It only cares that you can measure it with yes or no.

Here is my repository.

My name is Balasubramanyam Kosuri, and I work as a Technical writer. Connect with me on LinkedIn, for more such content.

Andrej Karpathy

Research

AI

Skills

Claude



Written by Balu Kosuri

166 followers · 23 following

Follow

Just keep writing

Responses (4)



Pancake Agree

What are your thoughts?



Andreaprato It
Apr 3 (edited)



Skill is not stateless. Every iteration have memory of the previus one, regardless of what you can do.



4

[Reply](#)



Chris Perrin he/him
5 days ago



let an AI agent run experiments on its own, measure the results, keep what works, throw away what doesn't, and repeat until it's good.

This is another fairly obvious AI usage that never occurred to me until now. Thanks!



3



1 reply

[Reply](#)



Rama Rao Desai
3 days ago



Being a 'Technical writer' is the closest thing to a programmer! You know the Technical jargons to use in the prompts. You know the architecture to be built! And finally you know how to iterate through the problems and what should the exact outcomes... [more](#)



1



1 reply

[Reply](#)

[See all responses](#)

More from the list: "Reading list"

Curated by Pancake Agree



Nitin Sharma

I Forced NotebookLM Into 6 Real Workflows —...

 · Apr 3



In Data Scie... by Sebastia...

RunAnywhere: Turning Your M2 Mac Into A...

 · Mar 11



In The Ai Stud... A

Clone An Functioni

 · Feb 11

[View list](#)

More from Balu Kosuri

 Balu Kosuri

Competitor Analysis Deep Research Prompts for...

A comprehensive collection of research prompts designed for AI-powered deep...

Feb 2



 Balu Kosuri

SEO, AEO, and GEO for Technical Writers

Imagine you have built the most beautiful library in the world. Every shelf is perfectl...

Mar 9



3



2



 Balu Kosuri

12 Prompting Techniques for Technical Writers

(with fill-in TEMPLATES)

Sep 11, 2025



4



1



 Balu Kosuri

Claude-Code Skills for Technical Writers

What are “skills”?

Mar 5



3



See all from Balu Kosuri

Recommended from Medium



Joe Njenga

**Claude Code Ultraplan
Launched: I Just Tested It (And...**

Anthropic has added Claude Code
ultraplan, and I was quick to test it. You...



6d ago



591



20



Michal Malewicz

Vibe Coding is OVER.

Here's What Comes Next.



Mar 24



4.8K



176



In Predict by Tasmia Sharmin

Palantir CEO Says Only Two Types Will Survive AI (And Elite...

Alex Karp told Gen Z there are “basically two ways to know you have a future.”...

★ Mar 26

👏 3.2K

💬 249

💬

⋮

Kristopher Dunham

Paperclip: The Open-Source Platform Turning AI Agents into...

Something strange happened when businesses started deploying AI agents a...

★ Mar 25

👏 326

💬 6

💬

⋮

In Level Up Coding by Fareed Khan

Building Claude Code with Harness Engineering

Multi-agents, MCP, skills system, context pipelines and more

★ 4d ago

👏 932

💬 6

💬

⋮

In The Ai Studio by Ai studio

11 Insane Use Cases of OpenClaw AI

What Happens When You Give an AI Agent Access to Your Entire Digital Life

★ Mar 27

👏 348

💬 11

💬

⋮

See more recommendations